PCB (Printed Circuit Board) Defect Detection

Contents

PCB (Printed Circuit Board) Defect Detection 1
What are PCB Defects? 1
PCB Image Files 2
PCB Defect Detection Program 2
Image Loading 2
Grayscale Conversion 3
Gaussian Blur 3
Difference Calculation 3
Thresholding 3
Labeling 4
Visualization 4
Bounding Boxes 4
Assignment: Adjust Threshold to Show all Defects 5
Assignment Submission 5

Time required: 60 minutes

What are PCB Defects?

PCB (Printed Circuit Boards) defect detection is a crucial process in ensuring the quality and reliability of printed circuit boards (PCBs). It involves identifying and classifying various types of defects that can occur during the manufacturing process.

Types of Defects

Common defects in PCBs include:

- **Missing Hole**: Absence of a required hole.
- **Mouse Bite**: Small, irregular cuts or nibbles on the edges.
- **Open Circuit**: Breaks in the conductive path.
- **Short Circuit**: Unintended connections between conductive paths.

- **Spur**: Unwanted protrusions of copper.
- **Spurious Copper**: Random copper deposits

Detection Methods

1. **Visual Inspection**: Manual inspection using microscopes or magnifying glasses.
2. **Automated Optical Inspection (AOI)**: Uses cameras and software to detect defects.
3. **X-ray Inspection**: Identifies internal defects not visible on the surface.
4. **Electrical Testing**: Checks for continuity and proper electrical connections

PCB Image Files

We are going to use Optical Inspection with MATLAB.

1. Download the following images.
    a. pcb_template.jpg is the template image.
    b. pcb_defect.jpb is the defect image.
2. Upload these files to MATLAB.

PCB Defect Detection Program

```
% Use image processing to detect PCB defects
clc % Clear the command window
```

Image Loading

- **Function**: imread
- **Purpose**: To load the images of the PCB boards into MATLAB.
- **Explanation**: The imread function reads an image file and stores it as a matrix in MATLAB. Each element of the matrix represents a pixel in the image.

```
%% Load the two PCB images
% Load the template PCB image (defect-free)
imageTemplate = imread('pcb_template.jpg');
% Load the test PCB image (potentially with defects)
imageDefect = imread('pcb_defect.jpg');
```

Grayscale Conversion

- **Function**: rgb2gray
- **Purpose**: To convert RGB images to grayscale.
- **Explanation**: The rgb2gray function converts a color image (RGB) to a grayscale image. Grayscale images are easier to process and analyze for defects.

```
% Convert images to grayscale
grayImageTemplate = rgb2gray(imageTemplate);
grayImageDefect = rgb2gray(imageDefect);
```

Gaussian Blur

- **Function**: imgaussfilt
- **Purpose**: To apply Gaussian blur to reduce noise.
- **Explanation**: The imgaussfilt function applies a Gaussian filter to smooth the image and reduce noise. This makes defect detection more accurate by minimizing irrelevant variations.

```
%% Convert images to grayscale
grayImageTemplate = rgb2gray(imageTemplate);
grayImageDefect = rgb2gray(imageDefect);
```

Difference Calculation

- **Function**: abs
- **Purpose**: To calculate the absolute difference between the two images.
- **Explanation**: The abs function computes the absolute value of the difference between corresponding pixels in the two images. This highlights areas where the images differ, which may indicate defects.

```
%% Calculate the absolute difference between the blurred images
diffImage = abs(blurredImageTemplate - blurredImageDefect);
```

Thresholding

- **Purpose**: To create a binary image that highlights defects.
- **Explanation**: Thresholding converts the difference image into a binary image, where pixels with values above a certain threshold are set to 1 (white), and others are set to 0 (black). This helps in isolating the defects. 50 is a good starting value.

```
%% Threshold the difference image to highlight defects
% Set a threshold value; lower values detect more defects
threshold = 50;
% Create binary image where pixels above the threshold are set to 1 (white
binaryDiffImage = diffImage > threshold;
```

Labeling

- **Function**: logical
- **Purpose**: To convert the binary image into a logical array for further processing.
- **Explanation**: The logical function converts the binary image into a logical array, where each element is either true (1) or false (0). This is useful for identifying and processing connected components (defects) in the image.

```
% Convert the binary image to a logical array
labeledImage = logical(binaryDiffImage);
```

Visualization

- **Functions**: figure, subplot, imshow, title
- **Purpose**: To display the original images and the binary difference image.
- **Explanation**: These functions create a figure window and display the images in subplots. imshow displays an image, and title adds a title to each subplot.

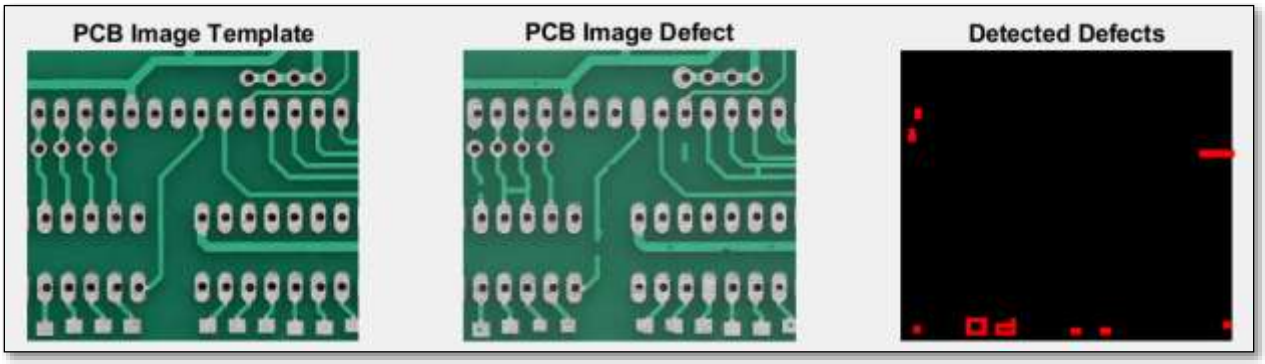
```
%% Display the original images and the difference image
figure; % Create a new figure window

subplot(1, 3, 1); % Create a subplot for the first image
imshow(imageTemplate); % Display the template PCB image
title('PCB Image 1'); % Add a title to the first subplot

subplot(1, 3, 2); % Create a subplot for the second image
imshow(imageDefect); % Display the test PCB image
title('PCB Image 2'); % Add a title to the second subplot

subplot(1, 3, 3); % Create a subplot for the difference image
imshow(binaryDiffImage); % Display the binary difference image
title('Detected Defects'); % Add a title to the third subplot
```

Bounding Boxes

- **Functions**: regionprops, rectangle
- **Purpose**: To draw bounding boxes around detected defects.

- **Explanation**: The regionprops function measures properties of labeled regions in the image, such as the bounding box.
- The rectangle function draws rectangles around these regions to highlight the defects.
  rectangle('Position', position, 'EdgeColor', color, 'LineWidth', width)

Example run:



Assignment: Adjust Threshold to Show all Defects

In this program, the threshold is 50. This is a good starting point. This shows most defects, but not all the defects.

What threshold value shows all defects, without showing false positives?

Assignment Submission

1. Submit properly named and commented script file.
2. Attach a text file or screenshot showing the successful execution of the script.
   a. Show a screenshot with your adjusted threshold value.
3. Attach all to the assignment in Blackboard.